Embedding Local Listener Script

1. Architectural Purpose & "The Why"

Traditional HTML web forms rely on a primitive submission layout: when a user clicks a button, the web page completely stops, forces a hard screen refresh, and flushes all current form fields down an instruction route. This completely breaks the user experience of modern applications, causing disruptive loading blinks and rendering bugs.

To mimic modern web application flows, this guide uses a client-side **Asynchronous Event Listener Loop**. Using vanilla JavaScript, the script runs in the background of the browser tab and watches your lead capture form form fields. When a user fires a submission, the script captures the entry parameters, pauses the browser's default reload sequence using event controllers, and forwards the variables cleanly down the server channel while keeping the interface completely smooth and responsive.

---

2. Complete JavaScript Source Code

Open your WordPress administration dashboard panel, go to **WPCode ➔ Header & Footer**, and paste this script container directly inside the **Footer** block text area.

html

```
<script>
document.addEventListener("DOMContentLoaded", function () {
    // Line 4: Locate the target lead registration form on the page template
    const leadForm = document.querySelector("#native-lead-form");
    const responseContainer = document.querySelector("#form-response-message");

    if (leadForm) {
        // Line 8: Attach the asynchronous event listener hook to trap form submissions
        leadForm.addEventListener("submit", function (event) {
            // Line 10: Halt default browser actions to block full-page reloads
            event.preventDefault();
```

```
            responseContainer.style.color = "#d97706"; // Amber processing
state
            responseContainer.innerText = "Synchronizing record with
system gateway...";

            // Line 15: Bundle text parameters into an internal request
envelope
            const formData = new FormData();
            formData.append('action', 'matecca_forward_lead'); // Hooks to
our backend plugin function
            formData.append('lead_name',
document.querySelector("#lead-name").value);
            formData.append('lead_email',
document.querySelector("#lead-email").value);

            // Line 21: Fetch the localized admin-ajax URL passed safely
by our plugin configuration
            const localGatewayUrl = typeof matecca_gateway !== 'undefined'
? matecca_gateway.ajax_url : '/wp-admin/admin-ajax.php';

            // Line 24: Post values strictly to your secure local back-end
server route
            fetch(localGatewayUrl, {
                method: 'POST',
                body: formData
            })
            .then(response => {
                if (!response.ok) throw new Error("Server communication
fault");
                return response.json();
            })
            .then(result => {
                // Line 33: Evaluate the structured response payload
returned by the PHP server gateway
                if (result.success) {
                    console.log("Enterprise Synchronization Success:",
result.data);
                    responseContainer.style.color = "#16a34a"; // Green
success code
                    responseContainer.innerText = `Success! Lead
processed. Score: ${result.data.score || 'Enriched'}`;
                    leadForm.reset(); // Wipe fields for next prospect
                } else {
                    throw new Error(result.data.message || "Internal
processing error");
                }
            })
```

```
            .catch(error => {
                console.error("CRM Gateway Pipeline Fault:", error);
                responseContainer.style.color = "#dc2626"; // Error red
                responseContainer.innerText = "Error syncing with system
gateway. View web console logs.";
            });
        });
    }
});
</script>
```

Use code with caution.

---

3. Line-by-Line Code Breakdown

- **Line 2 (`document.addEventListener("DOMContentLoaded", ...)`):** Ensures the script pauses execution until the web browser finishes building your page layout layout elements, preventing runtime script crashes due to missing form IDs.
- **Lines 4-5 (`querySelector`):** Reaches into your active browser window tree structure to acquire reference tags for your input forms (`#native-lead-form`) and your real-time screen message box (`#form-response-message`).
- **Line 8 (`leadForm.addEventListener("submit", ...)`):** Registers an active observer thread inside the user's browser. It continuously monitors the form container and springs to life the instant a user attempts to hit submit.
- **Line 10 (`event.preventDefault();`):** Blocks the core browser page-reload action. This allows the user to remain completely stationary on your layout while network transmissions process in the background.
- **Lines 15-18 (`new FormData()`):** Instantiates an empty virtual parameter package envelope. It appends the keyword indicator `matecca_forward_lead` to target your custom plugin, then grabs your text values directly from the input fields.
- **Line 21 (`typeof matecca_gateway !== 'undefined'`):** Scans the global browser variable workspace. If it finds the secure route map defined by your backend PHP plugin, it pulls that address; otherwise, it falls back safely to a default local sub-path.
- **Lines 24-27 (`fetch(localGatewayUrl, ...)`):** Launches an asynchronous background HTTP pipeline process. It pipes your form variables straight into your local server gateway link using an internal `POST` operation.

- **Lines 28-31 (`response.json()`):** Intercepts the raw data stream returning from your server layer and decodes it back into an active, readable JavaScript data object.
- **Lines 33-41 (`if (result.success)`):** Evaluates status variables written by your plugin. If success flags match, it modifies the screen text color parameter cleanly to green, echoes your custom Python CRM validation score on-screen, and runs `.reset()` to instantly clear the fields for the next entry.